Universidad Nacional Abierta y a Distancia

Vicerrectoría Académica y de Investigación

Unidad gestora: *Escuela*

Programa: *Ingeniería de Sistemas*

Curso: Big Data

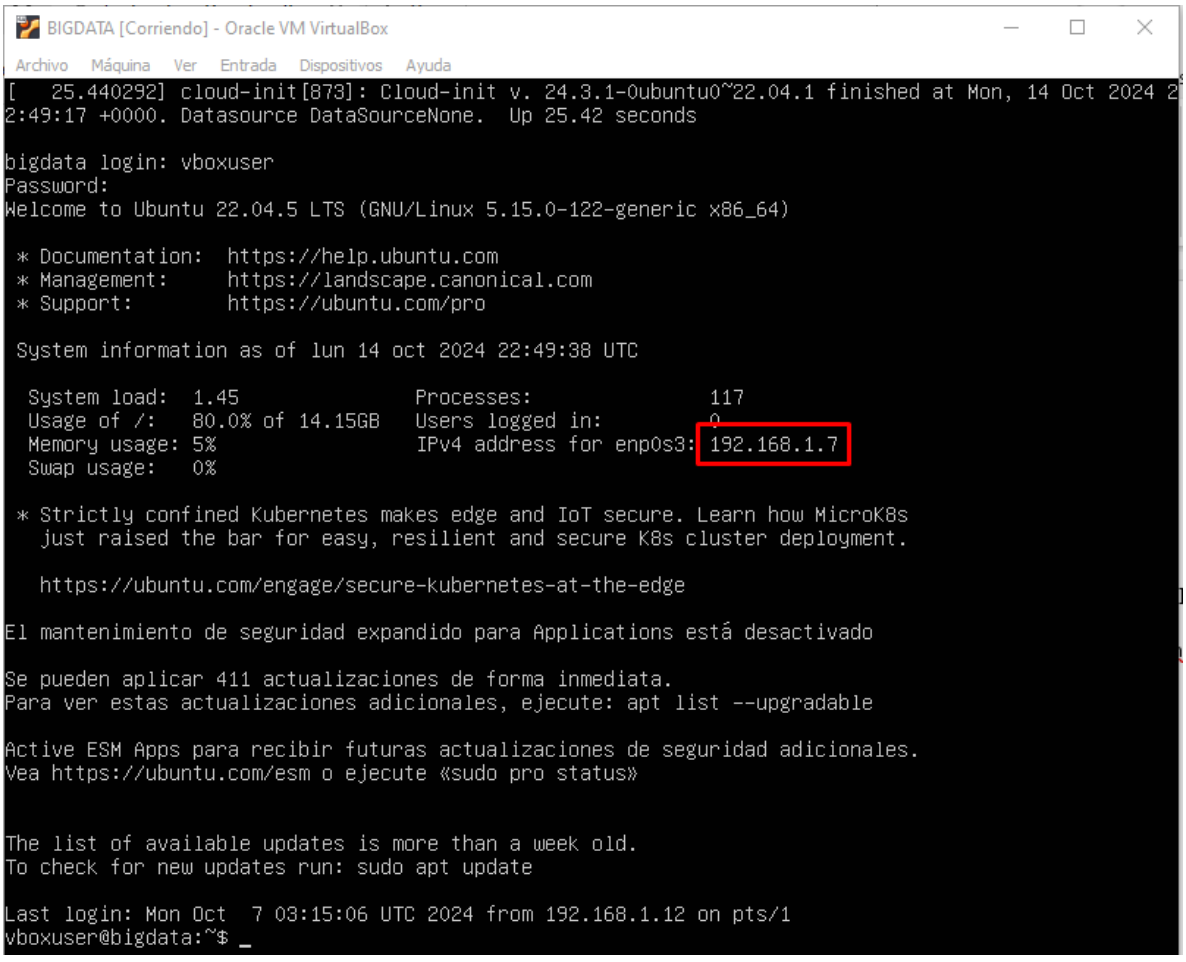
Código: 202016911

Análisis de Datos en Tiempo Real con Apache Kafka y Streaming

Ahora, realizaremos el proceso de configuración de Apache Spark y Streaming, para realizar el proceso de análisis de datos en tiempo real.

Figura 1.

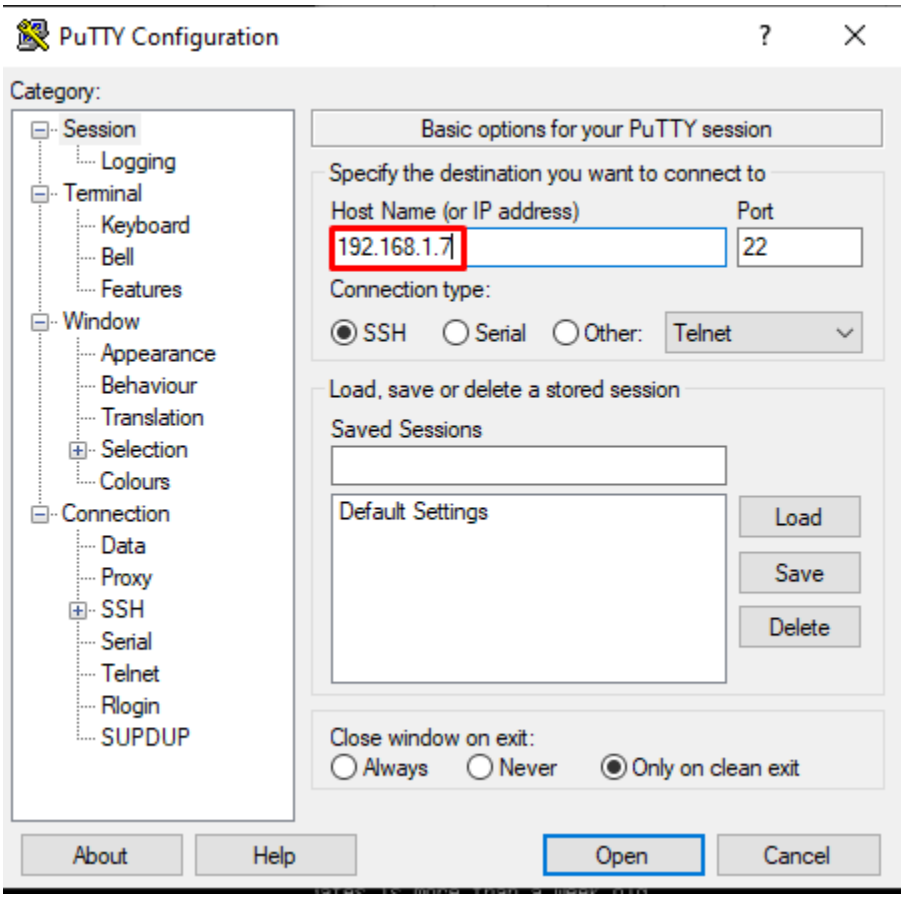
Inicio de sesión en VirtualBox



Nota: Se debe hacer uso de la máquina virtual configurada con hadoop y spark trabajada en las actividades anteriores. Iniciando sesión con el usuario “vboxuser” y contraseña “bigdata”.

Figura 2.

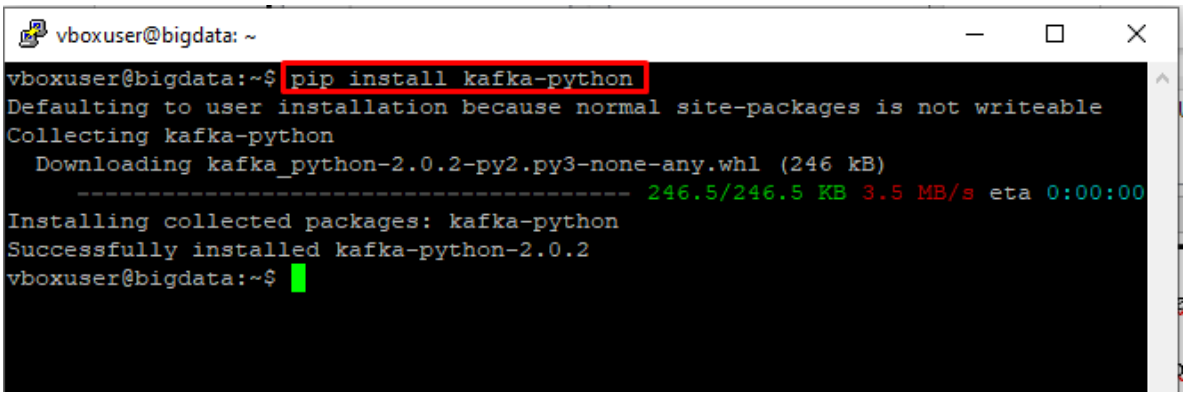
Inicio de sesión en Putty



Nota: Ejecutamos Putty para conectarnos por SSH a la máquina virtual utilizando la IP local que obtuvimos en el inicio de sesión en VirtualBox. En Putty iniciamos sesión con el usuario “vboxuser” y contraseña “bigdata”.

Figura 3.

Instalación de librería python en kafka



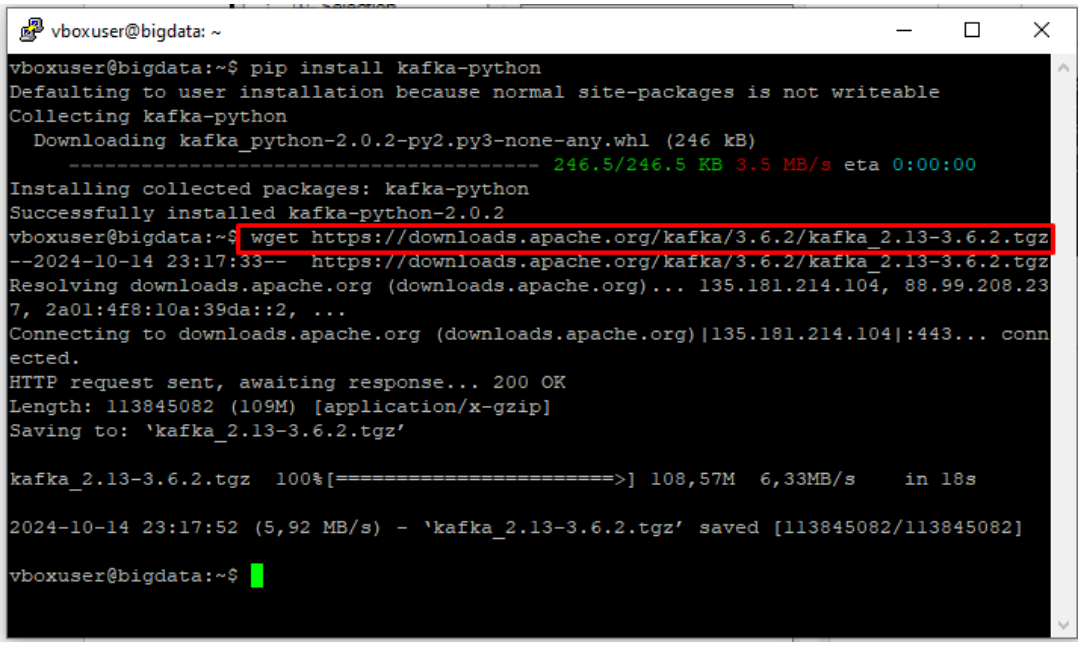
Nota: Realizamos el proceso de instalamos mediante PIP la librería de Python Kafka usando el comando “pip install kafka-python”

Descargue, descomprima y mueva de carpeta Apache Kafka

wget https://downloads.apache.org/kafka/3.9.2/kafka_2.12-3.9.2.tgz

Figura 4.

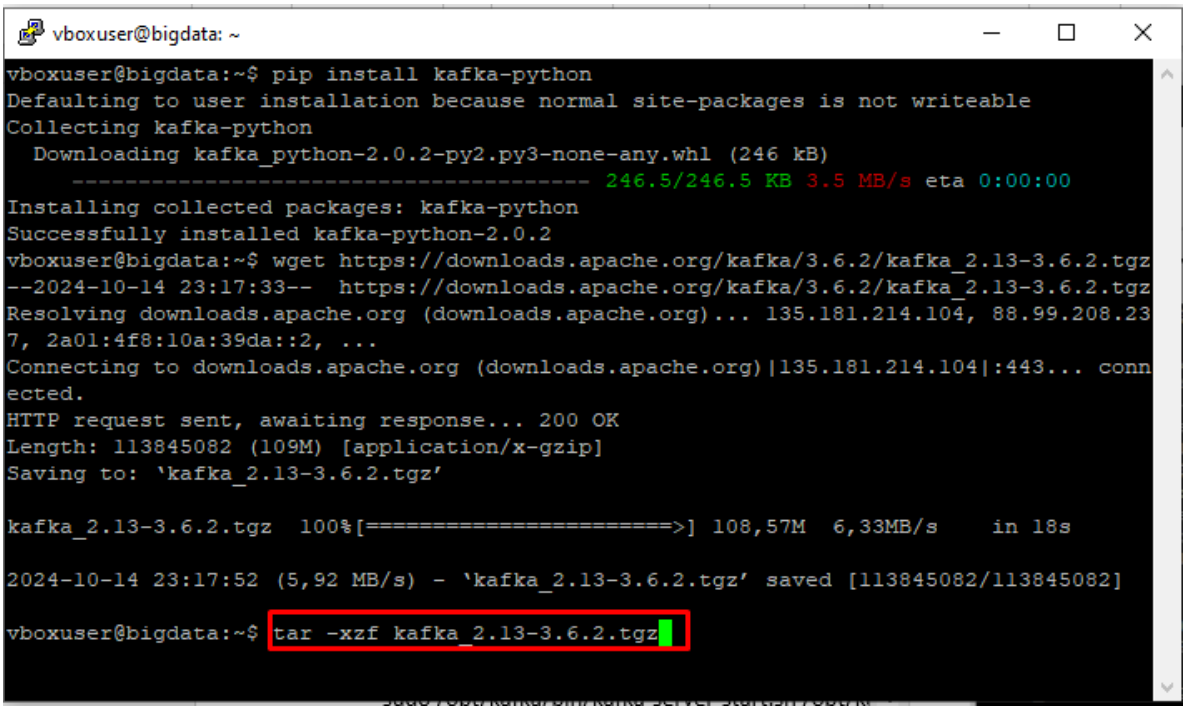
Descarga de apache kafka



Nota: Se realiza el proceso de descarga de Apache Kafka mediante el comando “wget https://downloads.apache.org/kafka/3.9.2/kafka_2.12-3.9.2.tgz”. Es importante validar que la versión 3.9.2 esté disponible en la página oficial https://downloads.apache.org/kafka/

Figura 5.

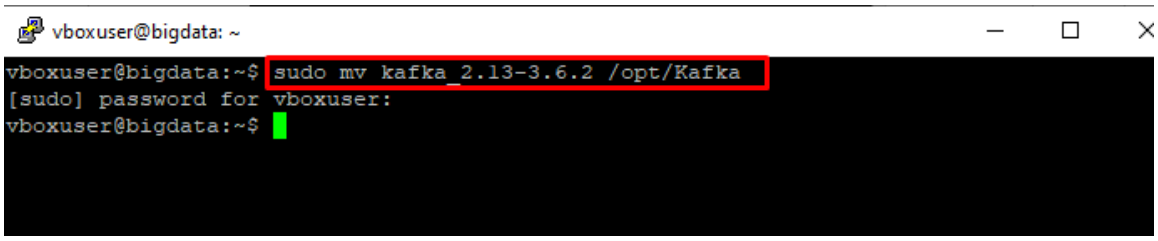
Descomprimir archivo de apache kafka



Nota: Se realiza el proceso de descomprimir el archivo de Apache Kafka descargada anteriormente mediante el comando “tar -xzf kafka_2.12-3.9.2.tgz”

Figura 6.

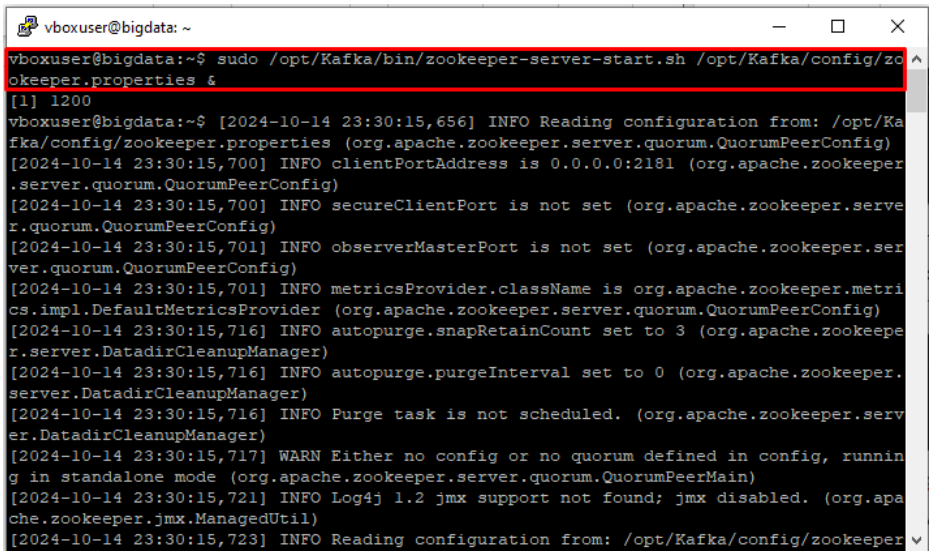
Renombrar archivo de apache kafka



Nota: Se realiza el proceso de cambiarle el nombre a el archivo descargado de Apache Kafka mediante el comando “sudo mv kafka_2.12-3.9.2 /opt/Kafka”. Si solicita contraseña “bigdata”

Figura 7.

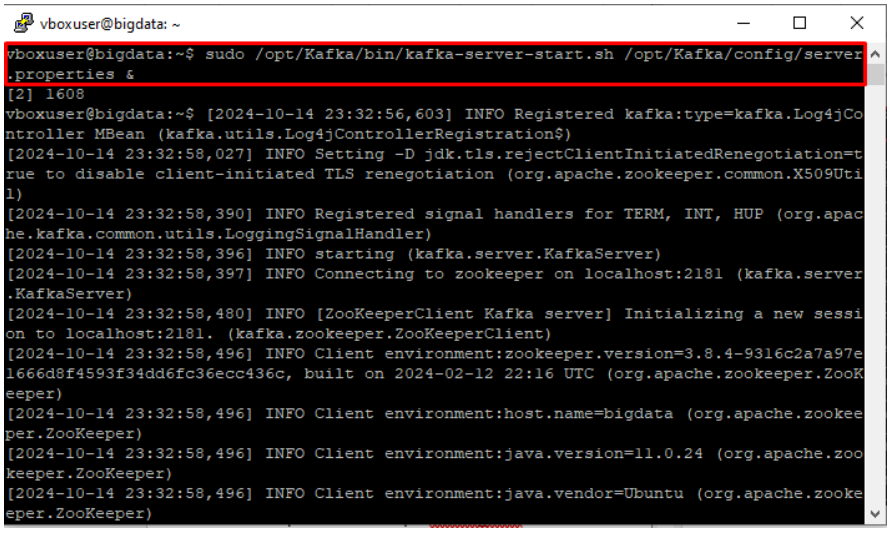
Inicio de servidor de ZooKeeper



Nota: Iniciamos el servidor de ZooKeeper mediante el comando “sudo /opt/Kafka/bin/zookeeper-server-start.sh /opt/Kafka/config/zookeeper.properties &”. Después de un momento y terminada la ejecución del comando anterior se debe dar Enter para que aparezca nuevamente el prompt del sistema.

Figura 8.

Inicio de servidor de apache kafka

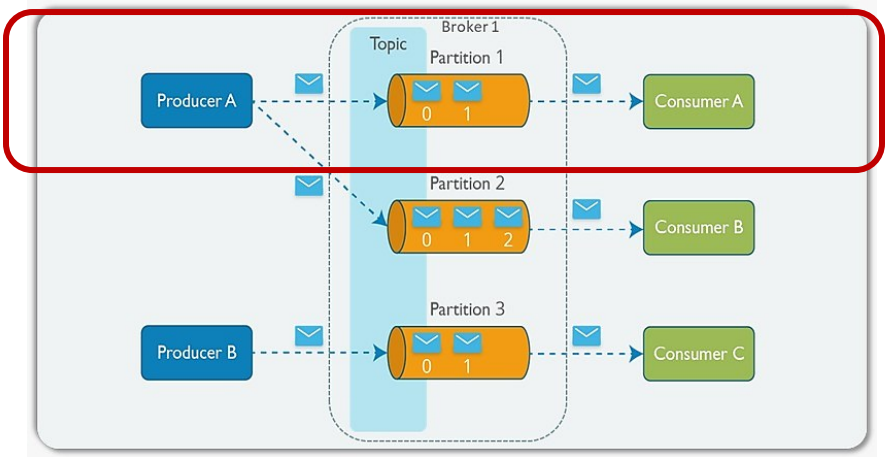


Nota: Iniciamos el servidor de Apache Kafka mediante el commando “sudo /opt/Kafka/bin/kafka-server-start.sh /opt/Kafka/config/server.properties &”.

Después de un momento y terminada la ejecución del comando anterior se debe dar Enter para que aparezca nuevamente el prompt del sistema.

Figura 9.

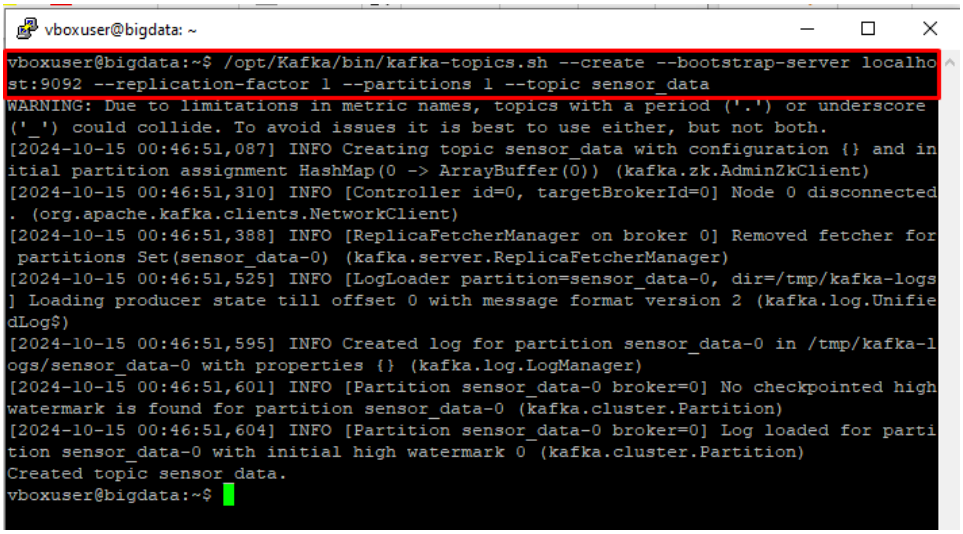
Ilustración de particionamiento de Kafka



Nota: Creamos un tema (topic) de Kafka, el tema se llamará sensor_data.

Figura 10.

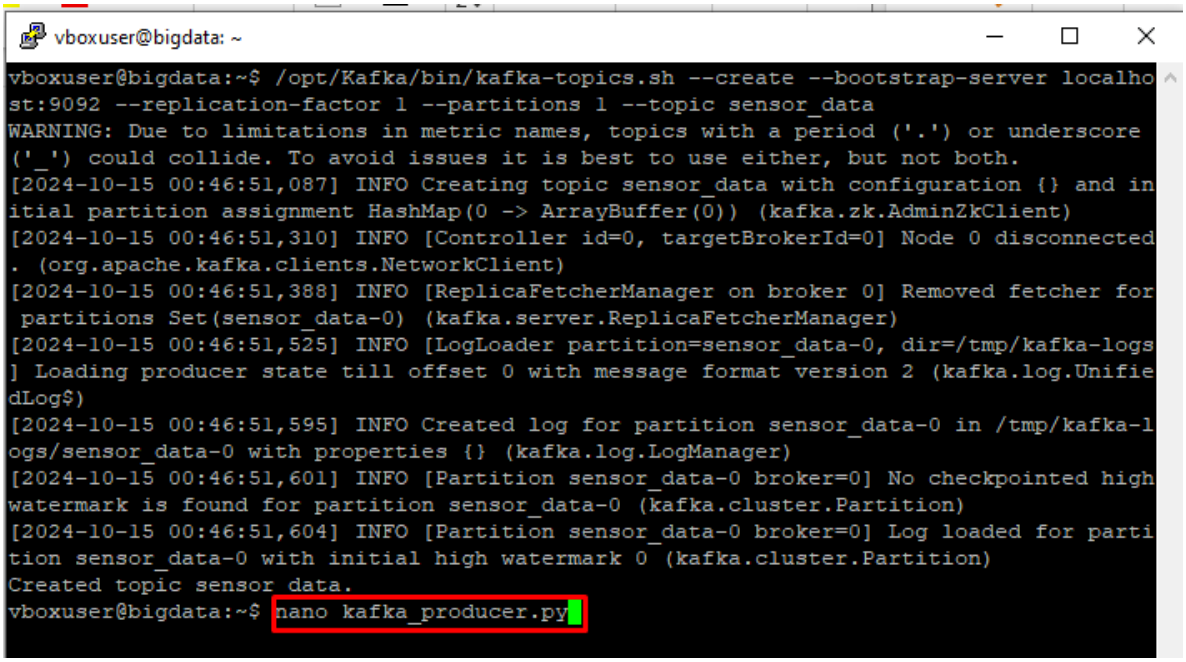
Creación de tema sensor_data



Nota: El tema sensor_data se crea mediante el siguiente código "/opt/Kafka/bin/kafka-topics.sh --create --bootstrap-server localhost:9092 --replication-factor 1 --partitions 1 --topic sensor_data"

Figura 11.

Implementación del producto de Kafka



Nota: Creamos un archivo llamado kafka_producer.py mediante el comando “nano kafka_producer.py”

Al archivo Kafka_producer agregamos las siguientes líneas de código:

```
import time
import json
import random
from kafka import KafkaProducer

def generate_sensor_data():
    return {
        "sensor_id": random.randint(1, 10),
        "temperature": round(random.uniform(20, 30), 2),
        "humidity": round(random.uniform(30, 70), 2),
        "timestamp": int(time.time())
    }

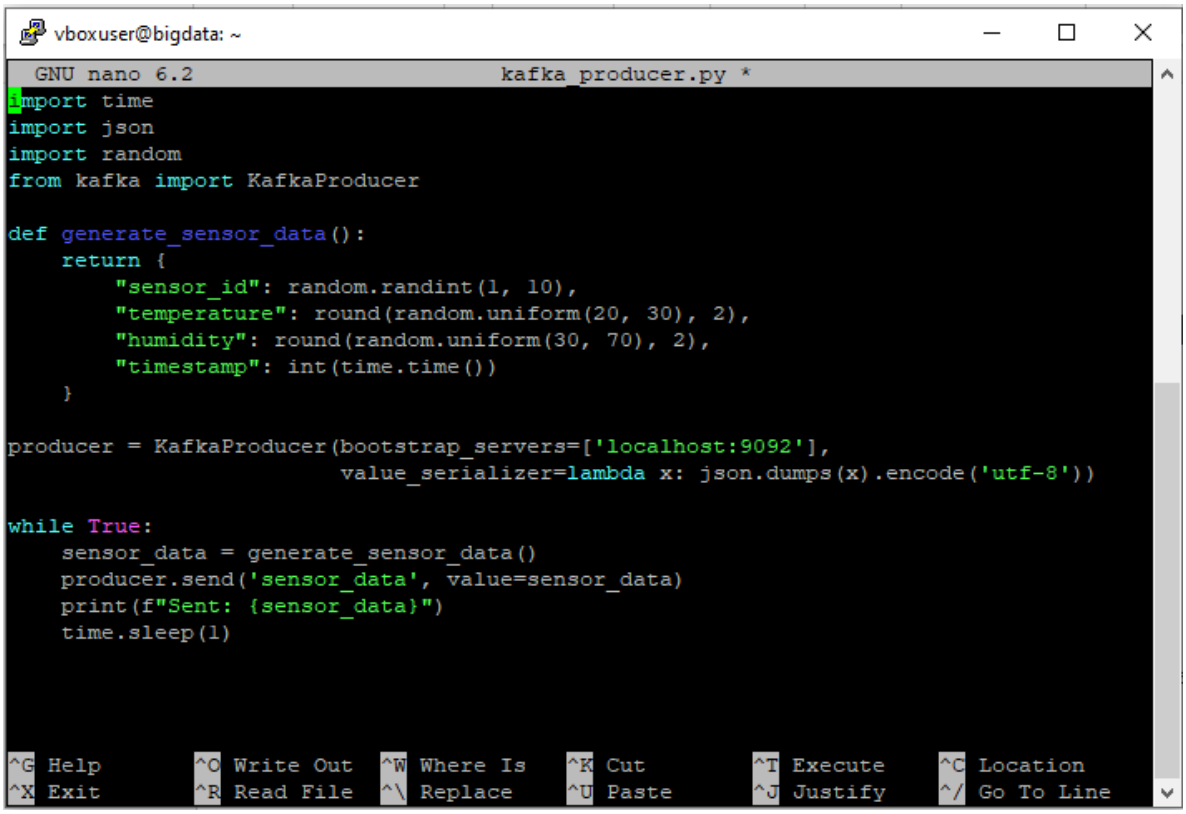
producer = KafkaProducer(bootstrap_servers=['localhost:9092'],
                         value_serializer=lambda x: json.dumps(x).encode('utf-8'))

while True:
```

```
  sensor_data = generate_sensor_data()
  producer.send('sensor_data', value=sensor_data)
  print(f"Sent: {sensor_data}")
  time.sleep(1)
```

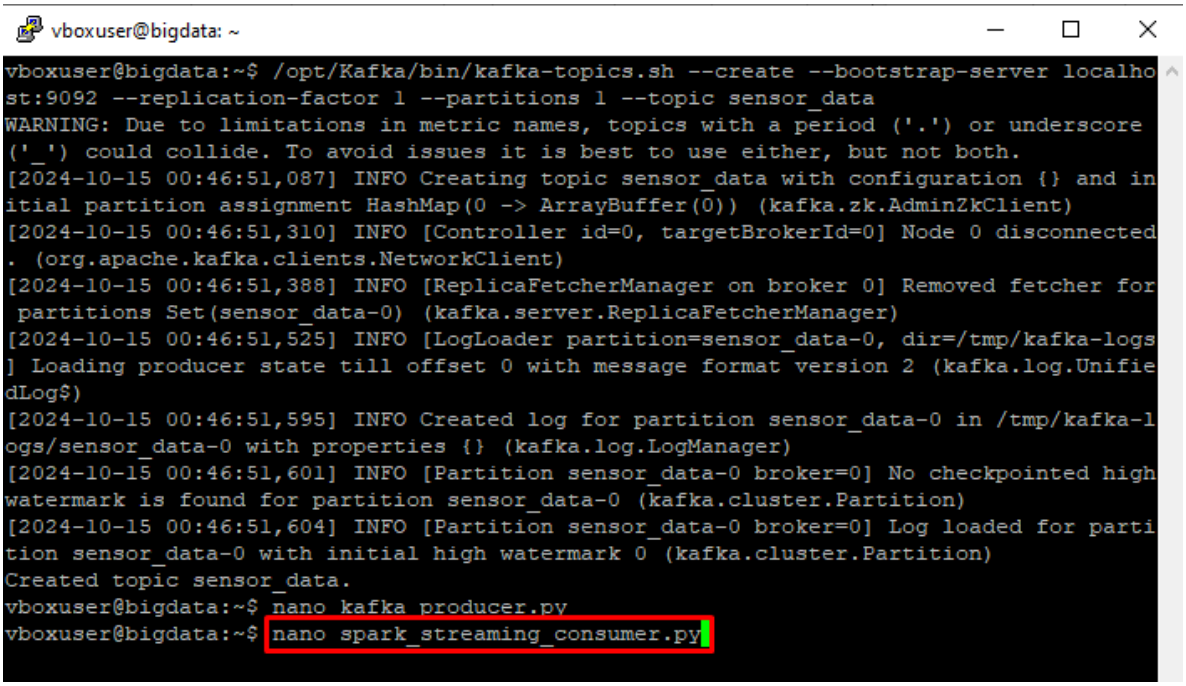
Figura 12.

Archivo kafka_producer.py



Nota: Dar Crtl+O enter y luego Crtl+X para salir. El código anterior genera datos simulados de sensores y los envía al tema (topic) de Kafka que creamos anteriormente (sensor_data).

Figura 13. Implementación del consumidor de spark streaming



Nota: Creamos un consumidor(consumer) utilizando Spark Streaming para procesar los datos en tiempo real. Creamos el archivo llamado spark_streaming_consumer.py por medio del comando “nano spark_streaming_consumer.py”.

Agregamos al archivo spark_streaming_consumer.py el siguiente codigo:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col, window
from pyspark.sql.types import StructType, StructField, IntegerType, FloatType,
TimestampType
import logging

# Configura el nivel de log a WARN para reducir los mensajes INFO
spark = SparkSession.builder \
    .appName("KafkaSparkStreaming") \
    .getOrCreate()

spark.sparkContext.setLogLevel("WARN")

# Definir el esquema de los datos de entrada
schema = StructType([
    StructField("sensor_id", IntegerType()),
    StructField("temperature", FloatType()),
    StructField("humidity", FloatType()),
    StructField("timestamp", TimestampType())
```

```
])

# Crear una sesión de Spark
spark = SparkSession.builder \
    .appName("SensorDataAnalysis") \
    .getOrCreate()

# Configurar el lector de streaming para leer desde Kafka
df = spark \
    .readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "localhost:9092") \
    .option("subscribe", "sensor_data") \
    .load()

# Parsear los datos JSON
parsed_df = df.select(from_json(col("value").cast("string"),
schema).alias("data")).select("data.*")

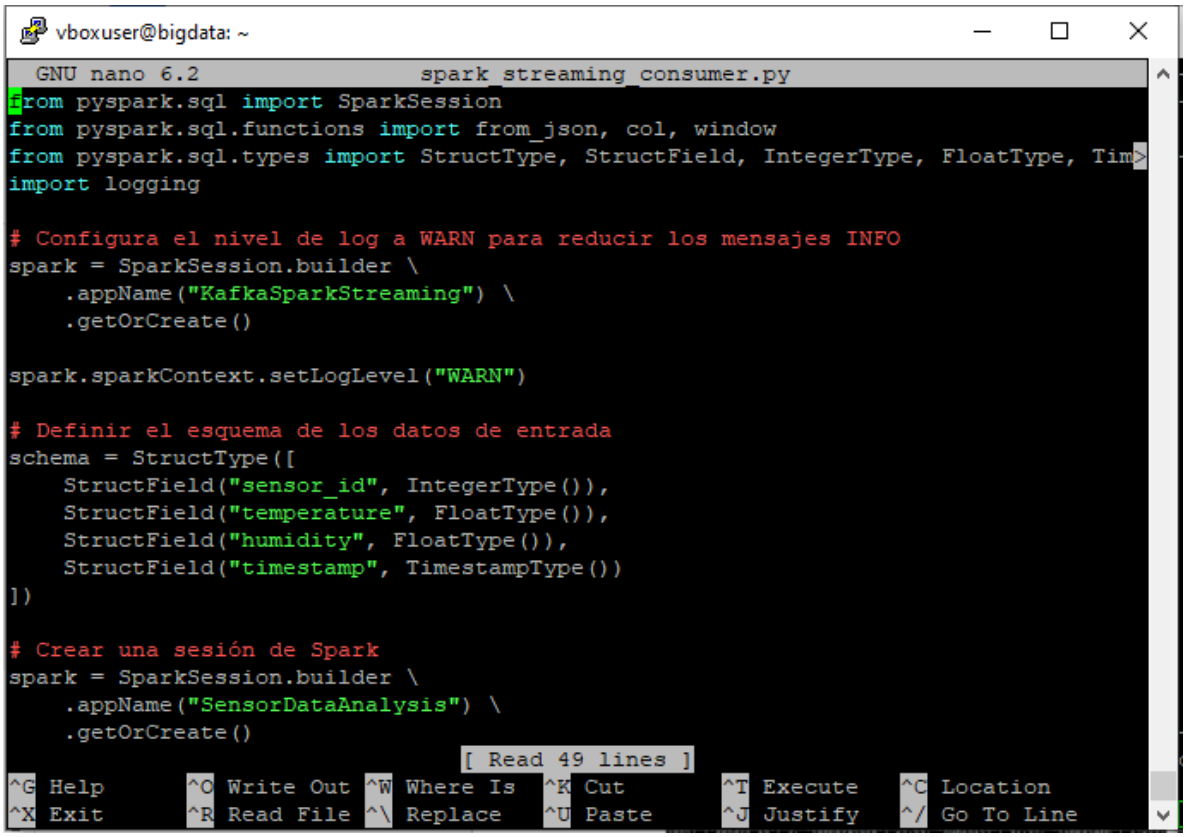
# Calcular estadísticas por ventana de tiempo
windowed_stats = parsed_df \
    .groupBy(window(col("timestamp"), "1 minute"), "sensor_id") \
    .agg({"temperature": "avg", "humidity": "avg"})

# Escribir los resultados en la consola
query = windowed_stats \
    .writeStream \
    .outputMode("complete") \
    .format("console") \
    .start()


query.awaitTermination()
```

Figura 14.

Ilustración de archivo spark_streaming_consumer.py

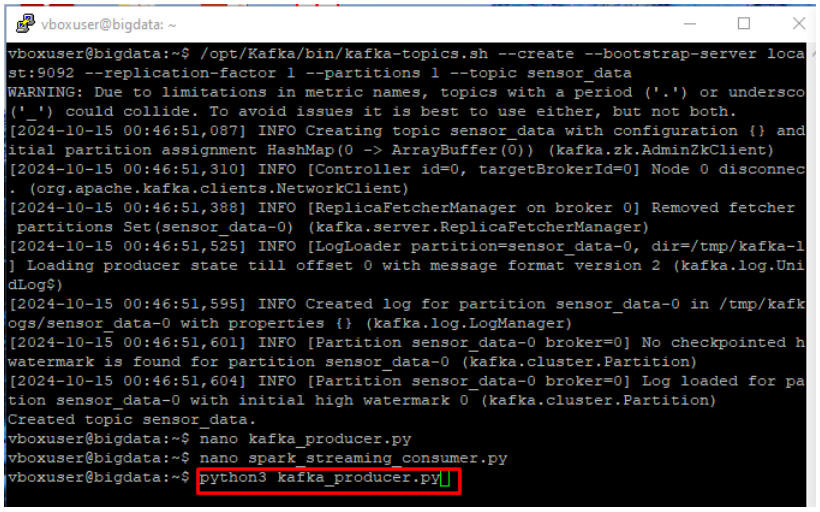


Nota: Dar Crtl+O enter y luego Crtl+X para salir. Este codigo utiliza Spark Streaming para leer datos del tema(topic) de Kafka, procesa los datos en ventanas de tiempo de 1 minuto y calcula la temperatura y humedad promedio para cada sensor.

Ejecución y análisis

Figura 15.

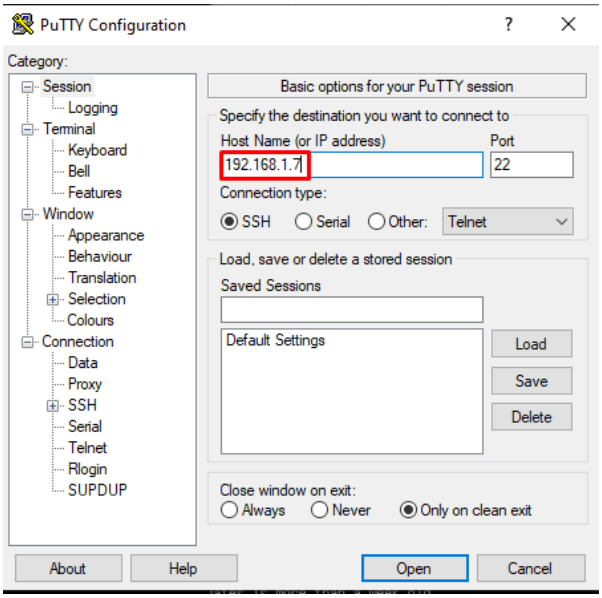
Ejecución de archivo kafka_producer.py



Nota: En una terminal, ejecutamos el productor(producer) de Kafka por medio del comando “python3 kafka_producer.py”.

Figura 16.

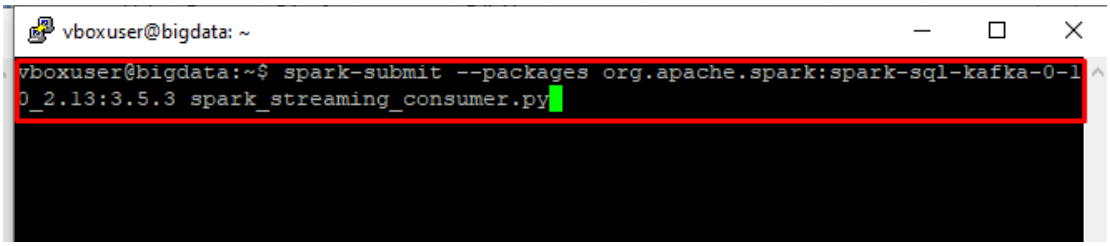
Nuevo Inicio de sesión en putty.



Nota: En otra terminal, ejecutamos el consumidor de Spark Streaming. Ejecutamos otra terminal de Putty para conectarnos por SSH a la máquina virtual utilizando la IP local sin cerrar la otra terminal de Putty. Y nos logueamos nuevamente usuario “vboxuser” y contraseña “bigdata”

Figura 17.

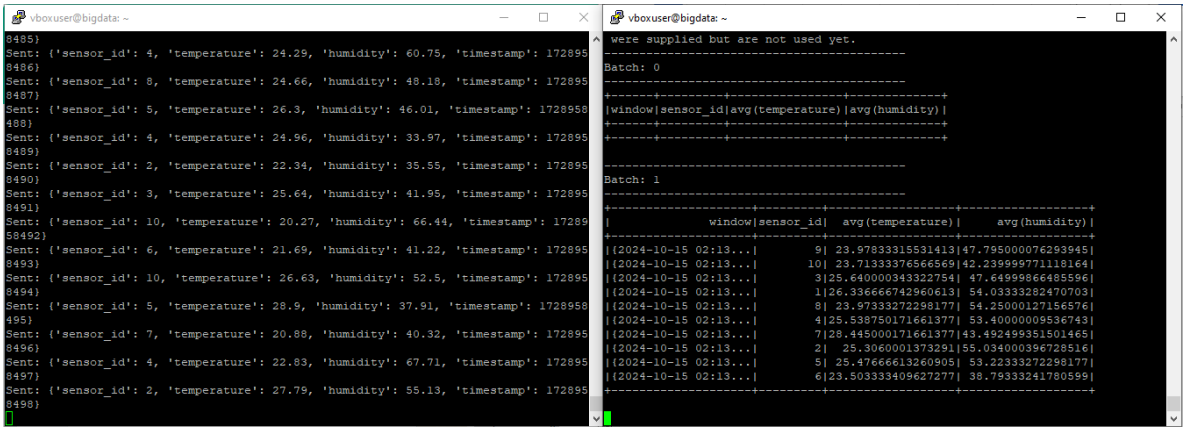
Ejecución del consumidor de spark streaming



Nota: Ejecutamos el consumidor de spark streaming mediante el comando “spark-submit --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.3 spark_streaming_consumer.py”.

Figura 18.

Ejecución del productor y análisis de datos del consumidor



Nota: Observamos la salida del productor y del consumidor de Spark Streaming y analizamos los resultados. Es importante primero ejecutar el script de productor y luego el del consumidor.

Podemos ver información sobre los Jobs, Stages, entre otros, realizados por Spark, para esto ingresamos a la consola web mediante la URL http://your-server-ip:4040. Para este caso http://192.168.1.7:4040

Figura 19.

Análisis de jobs en spark

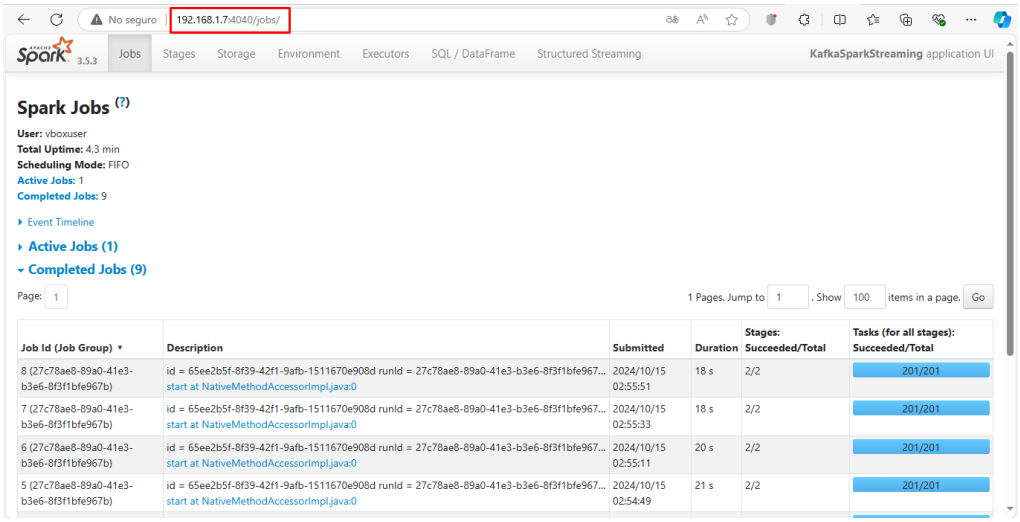


Figura 20.

Análisis de stages en spark

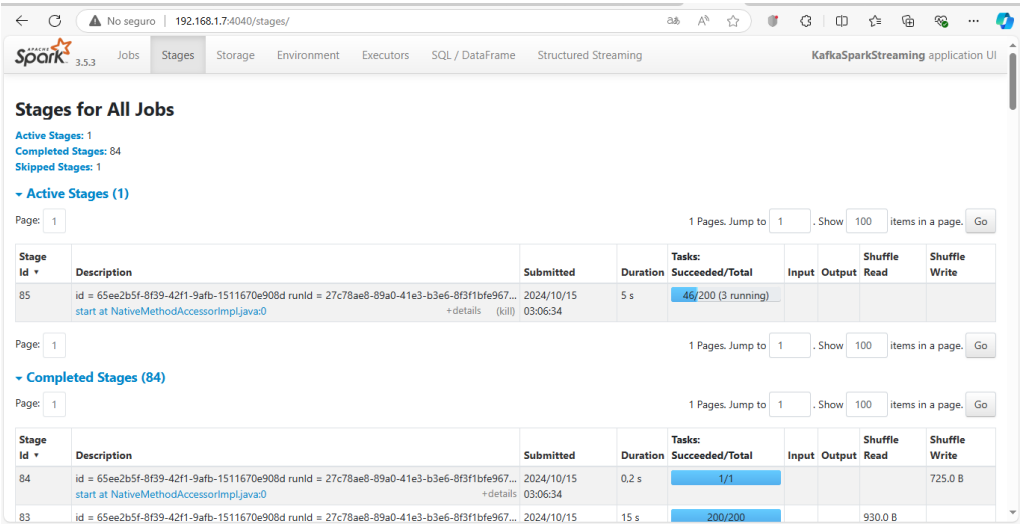


Figura 21.

Análisis de enviroment en spark

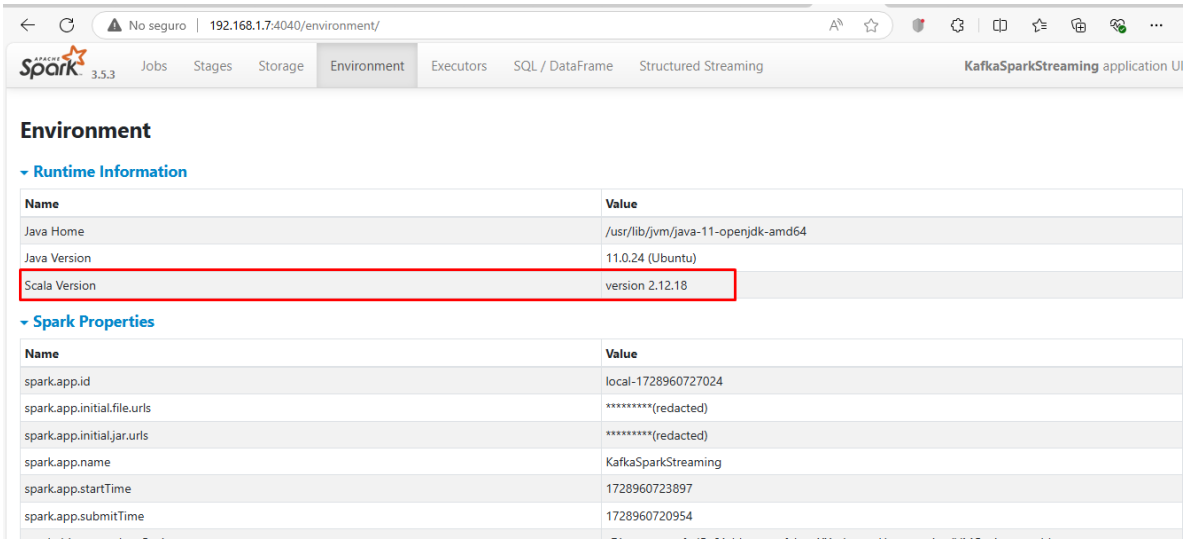
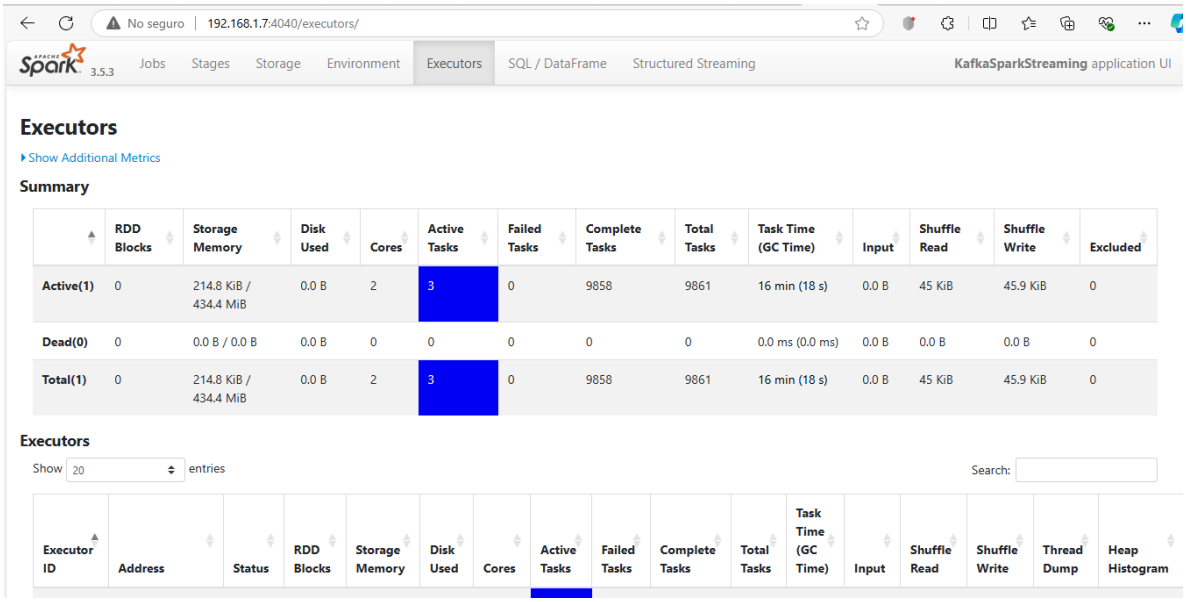


Figura 22.

Análisis de executors en spark



Nota: Para finalizar la ejecución de los scripts en Putty damos CTRL+C